

Lab4 实验报告

李毅PB22051031

1. 实验测试batch_size是不是越大越好？可能原因有哪些？

基于提供的baseline 模型进行训练。epochs=2

(1)batch_size=64

```
Accuracy of the network on the 10000 test images: 28 %
Accuracy for class: plane is 49.9 %
Accuracy for class: car is 25.9 %
Accuracy for class: bird is 4.0 %
Accuracy for class: cat is 7.3 %
Accuracy for class: deer is 4.0 %
Accuracy for class: dog is 37.2 %
Accuracy for class: frog is 70.0 %
Accuracy for class: horse is 18.6 %
Accuracy for class: ship is 25.8 %
Accuracy for class: truck is 42.3 %
```

(2)batch_size=128

```
Accuracy of the network on the 10000 test images: 15 %
Accuracy for class: plane is 3.0 %
Accuracy for class: car is 7.9 %
Accuracy for class: bird is 0.0 %
Accuracy for class: cat is 0.0 %
Accuracy for class: deer is 33.5 %
Accuracy for class: dog is 12.0 %
Accuracy for class: frog is 0.0 %
Accuracy for class: horse is 0.1 %
Accuracy for class: ship is 88.9 %
Accuracy for class: truck is 7.9 %
```

(3)batch_size=32

```
Accuracy of the network on the 10000 test images: 39 %
Accuracy for class: plane is 51.8 %
Accuracy for class: car is 53.2 %
Accuracy for class: bird is 14.6 %
Accuracy for class: cat is 17.5 %
Accuracy for class: deer is 23.4 %
Accuracy for class: dog is 36.1 %
Accuracy for class: frog is 58.3 %
Accuracy for class: horse is 51.6 %
Accuracy for class: ship is 33.7 %
Accuracy for class: truck is 50.5 %
```

batch_size增大可以增加吞吐量，提升训练速度，提升收敛速度，但更新步数减少：同样的 epoch 数下，batch_size 越大，迭代次数（参数更新次数）越少，模型收敛不足，模型泛化能力下降。

2. 在训练集那里的transform试一下RandomHorizontalFlip，效果会更好吗？

增加RandomHorizontalFlip()，保持batch_size=64。epochs=2

```
# 1. 定义数据预处理流水线：训练集加随机水平翻转，测试集不做增强
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),           # 随机水平翻转
    transforms.ToTensor(),                       # 转为 Tensor 并归一化到
    [0,1]                                         # 标准化到 [-1,1]
    transforms.Normalize((0.5, 0.5, 0.5),
                          (0.5, 0.5, 0.5))
])
test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                          (0.5, 0.5, 0.5))
])

# 2. 加载训练集并创建 DataLoader (使用 train_transform)
trainset = torchvision.datasets.CIFAR10(
    root=data_root, train=True, download=download, transform=train_transform
)
trainloader = torch.utils.data.DataLoader(
    trainset, batch_size=batch_size, shuffle=True, num_workers=2
)

# 3. 加载测试集并创建 DataLoader (使用 test_transform)
testset = torchvision.datasets.CIFAR10(
    root=data_root, train=False, download=download, transform=test_transform
)
testloader = torch.utils.data.DataLoader(
    testset, batch_size=batch_size, shuffle=False, num_workers=2
)
```

三次运行结果如下：

```
Accuracy of the network on the 10000 test images: 30 %
Accuracy for class: plane is 47.4 %
Accuracy for class: car is 40.8 %
Accuracy for class: bird is 1.9 %
Accuracy for class: cat is 0.8 %
Accuracy for class: deer is 14.1 %
Accuracy for class: dog is 30.3 %
Accuracy for class: frog is 52.1 %
Accuracy for class: horse is 43.8 %
Accuracy for class: ship is 34.3 %
Accuracy for class: truck is 41.3 %

Accuracy of the network on the 10000 test images: 27 %
Accuracy for class: plane is 46.4 %
Accuracy for class: car is 17.3 %
Accuracy for class: bird is 17.0 %
Accuracy for class: cat is 0.6 %
Accuracy for class: deer is 1.0 %
Accuracy for class: dog is 28.9 %
```

```
Accuracy for class: frog is 37.2 %
Accuracy for class: horse is 41.4 %
Accuracy for class: ship is 44.5 %
Accuracy for class: truck is 41.5 %
```

```
Accuracy of the network on the 10000 test images: 29 %
Accuracy for class: plane is 47.6 %
Accuracy for class: car is 40.4 %
Accuracy for class: bird is 3.1 %
Accuracy for class: cat is 2.0 %
Accuracy for class: deer is 10.5 %
Accuracy for class: dog is 25.9 %
Accuracy for class: frog is 49.3 %
Accuracy for class: horse is 38.1 %
Accuracy for class: ship is 48.9 %
Accuracy for class: truck is 27.1 %
```

可以看出，提升不大

3. 换一个optimizer, 使效果更好一些?

保持epochs=2

```
optimizer = optim.Adam(net.parameters(), lr=0.001)
```

结果如下:

```
Accuracy of the network on the 10000 test images: 52 %
Accuracy for class: plane is 59.0 %
Accuracy for class: car is 59.5 %
Accuracy for class: bird is 40.7 %
Accuracy for class: cat is 38.1 %
Accuracy for class: deer is 26.2 %
Accuracy for class: dog is 39.2 %
Accuracy for class: frog is 76.4 %
Accuracy for class: horse is 65.4 %
Accuracy for class: ship is 53.6 %
Accuracy for class: truck is 64.6 %
```

显著提升

4. 保持epoch数不变, 加一个scheduler, 是否能让效果更好一些?

```
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)

# 每个 epoch 结束后, 更新 lr
scheduler.step()
current_lr = scheduler.get_last_lr()[0]
print(f'>>> Epoch {epoch+1} done. Learning rate is now {current_lr:.6f}')
```

使用Adam, 跑20个epoch. 未使用scheduler时的准确率如下:

```
Accuracy of the network on the 10000 test images: 68 %
Accuracy for class: plane is 76.1 %
Accuracy for class: car is 82.2 %
Accuracy for class: bird is 52.5 %
Accuracy for class: cat is 51.4 %
Accuracy for class: deer is 71.4 %
Accuracy for class: dog is 55.5 %
Accuracy for class: frog is 74.4 %
Accuracy for class: horse is 72.2 %
Accuracy for class: ship is 78.3 %
Accuracy for class: truck is 72.5 %
```

使用scheduler，每10个epoch，学习率乘以0.1，结果如下：

```
Accuracy of the network on the 10000 test images: 67 %
Accuracy for class: plane is 71.9 %
Accuracy for class: car is 80.2 %
Accuracy for class: bird is 55.1 %
Accuracy for class: cat is 49.1 %
Accuracy for class: deer is 62.0 %
Accuracy for class: dog is 51.5 %
Accuracy for class: frog is 75.3 %
Accuracy for class: horse is 74.3 %
Accuracy for class: ship is 79.9 %
Accuracy for class: truck is 74.6 %
```

没有显著提升。

5. 根据Net() 生成 Net1(), 加入三个batch_normalization层, 显示测试结果, 效果是否更好?

别的设置同题4，使用scheduler

Net1类如下：

```
class Net1(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.bn1 = nn.BatchNorm2d(6)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.bn2 = nn.BatchNorm2d(16)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.bn3 = nn.BatchNorm1d(120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = F.relu(x)
        x = self.pool(x)
```

```

x = self.conv2(x)
x = self.bn2(x)
x = F.relu(x)
x = self.pool(x)

x = torch.flatten(x, 1)

x = self.fc1(x)
x = self.bn3(x)
x = F.relu(x)

x = F.relu(self.fc2(x))
x = self.fc3(x)
return x

```

训练结果如下:

```

Accuracy of the network on the 10000 test images: 69 %
Accuracy for class: plane is 76.0 %
Accuracy for class: car is 81.3 %
Accuracy for class: bird is 59.0 %
Accuracy for class: cat is 48.6 %
Accuracy for class: deer is 65.3 %
Accuracy for class: dog is 55.6 %
Accuracy for class: frog is 80.0 %
Accuracy for class: horse is 73.1 %
Accuracy for class: ship is 78.8 %
Accuracy for class: truck is 77.8 %

```

没有显著提升。

6. 根据Net() 生成Net2(), 使用Kaiming初始化卷积与全连接层, 显示测试结果, 效果是否更好?

Net2类如下:

```

class Net2(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    # 使用 kaiming 初始化卷积与全连接层
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_in',
nonlinearity='relu')
            if m.bias is not None:
                nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.Linear):

```

```

        nn.init.kaiming_normal_(m.weight, mode='fan_in',
nonlinearity='relu')
        if m.bias is not None:
            nn.init.constant_(m.bias, 0)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

```

训练结果如下:

```

Accuracy of the network on the 10000 test images: 66 %
Accuracy for class: plane is 72.2 %
Accuracy for class: car is 79.8 %
Accuracy for class: bird is 49.3 %
Accuracy for class: cat is 46.3 %
Accuracy for class: deer is 63.0 %
Accuracy for class: dog is 52.4 %
Accuracy for class: frog is 78.8 %
Accuracy for class: horse is 71.9 %
Accuracy for class: ship is 80.2 %
Accuracy for class: truck is 70.0 %

```

没有显著提升。

7. 根据Net()生成Net3(),将Net()中的通道数加到原来的2倍, 显示测试结果, 效果是否更好?

Net3类如下:

```

class Net3(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 12, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(12, 32, 5)
        self.fc1 = nn.Linear(32 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

```

训练结果如下：

```
Accuracy of the network on the 10000 test images: 72 %
Accuracy for class: plane is 76.9 %
Accuracy for class: car is 84.2 %
Accuracy for class: bird is 59.6 %
Accuracy for class: cat is 55.3 %
Accuracy for class: deer is 61.4 %
Accuracy for class: dog is 59.0 %
Accuracy for class: frog is 83.5 %
Accuracy for class: horse is 76.4 %
Accuracy for class: ship is 83.9 %
Accuracy for class: truck is 81.1 %
```

准确率小幅提升

8. 在不改变Net()的基础结构（卷积层数、全连接层数不变）和训练epoch数的前提下，你能得到最好的结果是多少？

最好效果为T7，准确率可以达到70%出头。

9. 使用ResNet18(),显示测试结果，可以分别测试不使用预训练权重和使用预训练权重

ResNet18类如下：

```
class ResNet18(nn.Module):
    def __init__(self, pretrained=False, num_classes=10):
        super().__init__()
        self.model = resnet18(pretrained=pretrained)
        self.model.fc = nn.Linear(self.model.fc.in_features, num_classes) # 修改最后的全连接层

    def forward(self, x):
        return self.model(x)
```

不使用预训练权重结果：

```
Accuracy of the network on the 10000 test images: 81 %
Accuracy for class: plane is 83.0 %
Accuracy for class: car is 90.0 %
Accuracy for class: bird is 74.4 %
Accuracy for class: cat is 62.8 %
Accuracy for class: deer is 81.6 %
Accuracy for class: dog is 71.1 %
Accuracy for class: frog is 86.6 %
Accuracy for class: horse is 85.1 %
Accuracy for class: ship is 87.8 %
Accuracy for class: truck is 88.0 %
```

使用预训练权重结果：

Accuracy of the network on the 10000 test images: 84 %

Accuracy for class: plane is 87.7 %

Accuracy for class: car is 91.6 %

Accuracy for class: bird is 79.7 %

Accuracy for class: cat is 69.4 %

Accuracy for class: deer is 85.5 %

Accuracy for class: dog is 73.9 %

Accuracy for class: frog is 89.8 %

Accuracy for class: horse is 89.6 %

Accuracy for class: ship is 89.6 %

Accuracy for class: truck is 87.5 %